D15A- 04

# ML-DL Experiment No. 9

## To create a simple Long Short-Term Memory (LSTM) network to forecast time series data using Deep Network Designer

## Dataset Source

Dataset: Sunspot Time Series Dataset

Provided within: MATLAB

Reference:
https://in.mathworks.com/help/deeplearning/gs/get-started-with-time-series-forecasting.html

### 1. Dataset Description

The dataset consists of historical sunspot observations recorded over time, representing a time series.

Dataset Characteristics

| Feature | Description |
|---|---|
| Data Type | Time series |
| Input Feature | Sunspot value |
| Time Steps | Sequential observations |
| Nature | Continuous numerical data |
| Prediction Type | Regression |

**Input Feature**

- Single variable: Sunspot intensity values over time

**Target Variable**

- Next time step value of sunspot data

**Key Characteristics**

- Sequential dependency between observations
- Suitable for temporal modeling
- Requires sequence-based input format

**2. Mathematical Formulation of LSTM**

1. Forget Gate

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

2. Input Gate

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

3. Cell State Update

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

4. Output Gate

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

5. Hidden State

$$h_t = o_t \cdot \tanh(C_t)$$

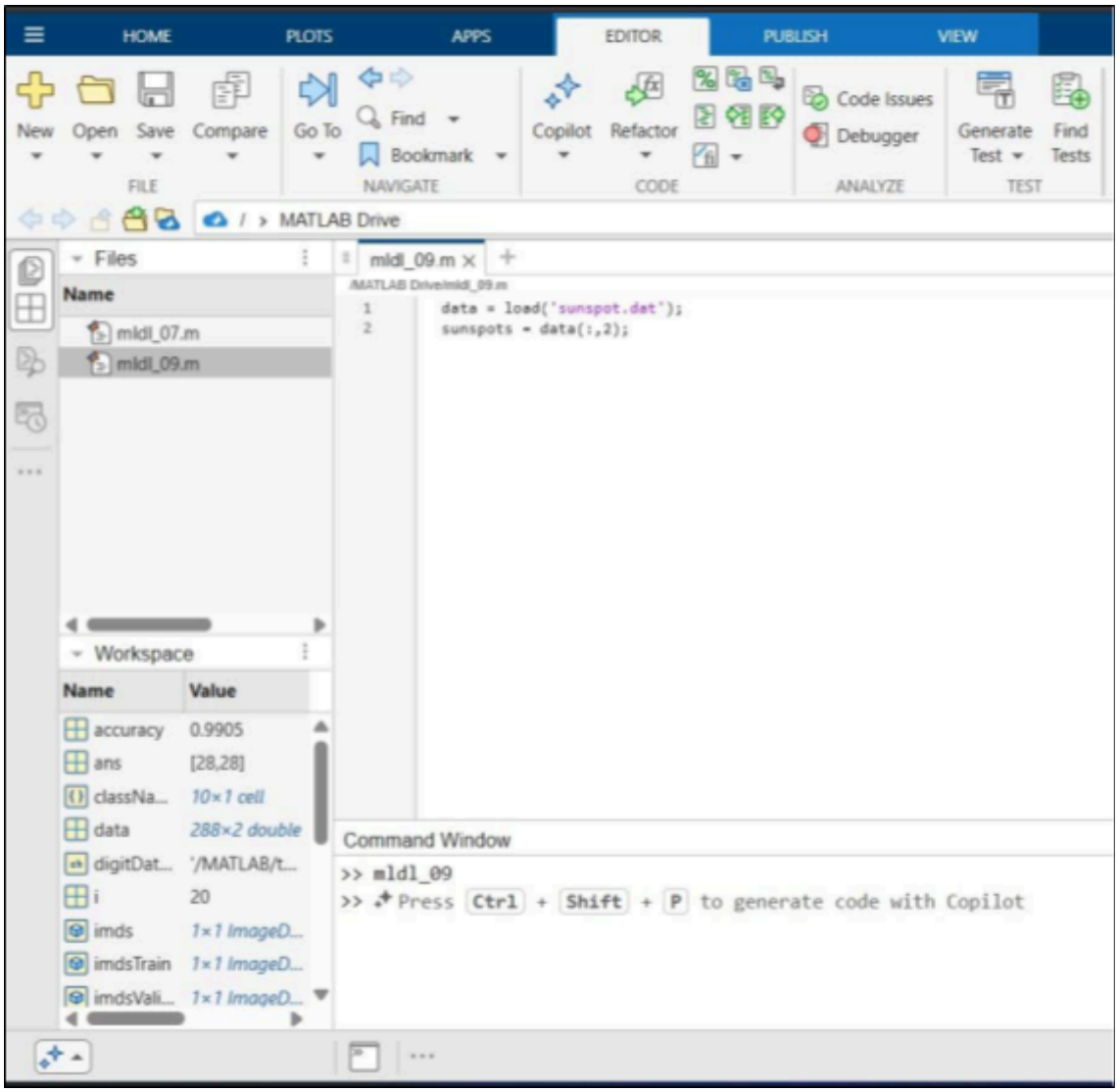
## 3. Algorithm Limitations

- Requires large sequential data for better learning
- Training is computationally expensive
- Sensitive to hyperparameters
- Difficult to interpret internal memory states
- Can overfit on small datasets
- Slower compared to simple models

## 4. Methodology / Workflow

### Steps Performed

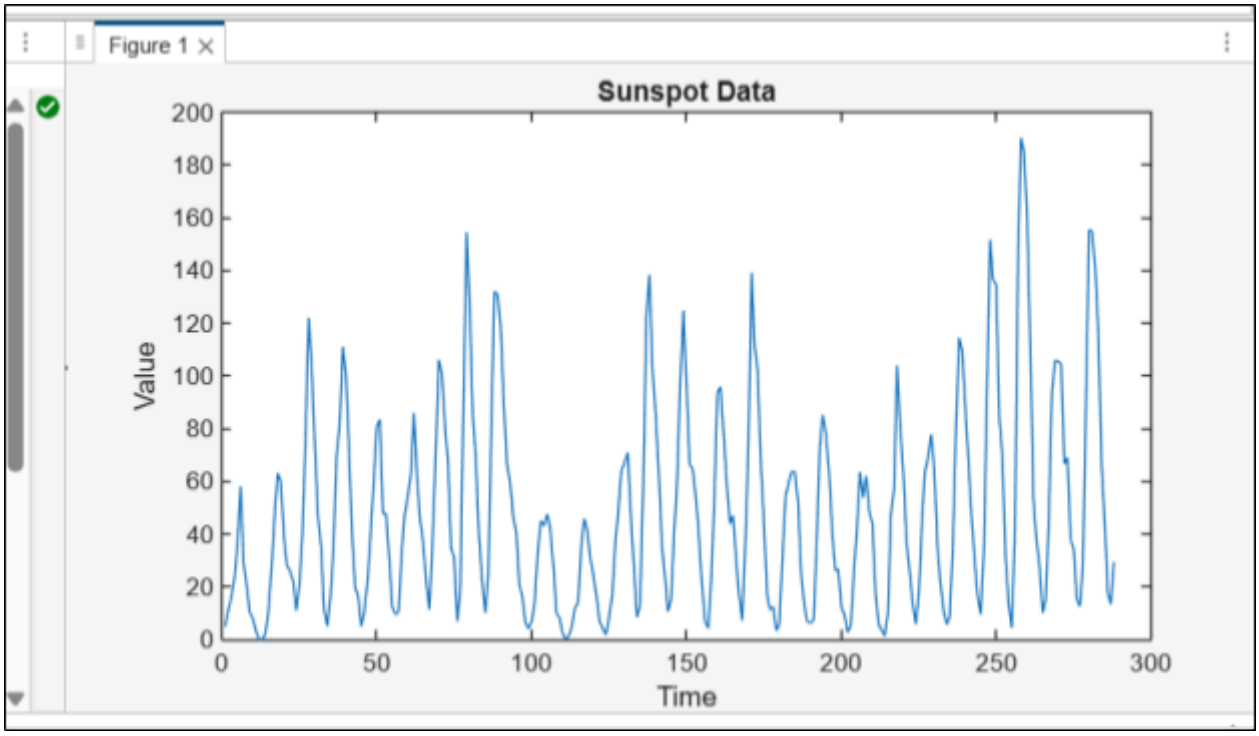
1. Load sunspot dataset in MATLAB




```
%% Step 1: Load Dataset
data = load('sunspot.dat');
sunspots = data(:,2);
```

2. Visualize time series data

```
plot(sunspots)
title('Sunspot Data')
xlabel('Time')
ylabel('Value')
```



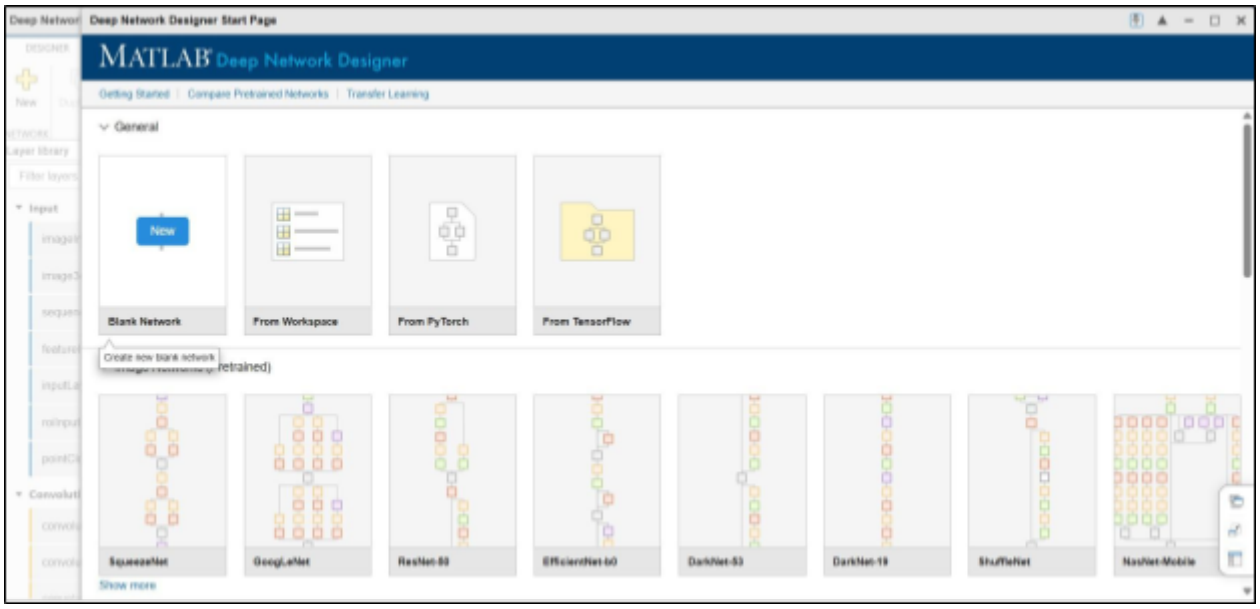

3. Normalize dataset for stable training

```
%% Step 3: Normalize Data
mu = mean(sunspots);
sigma = std(sunspots);
sunspots = (sunspots - mu) / sigma;
```

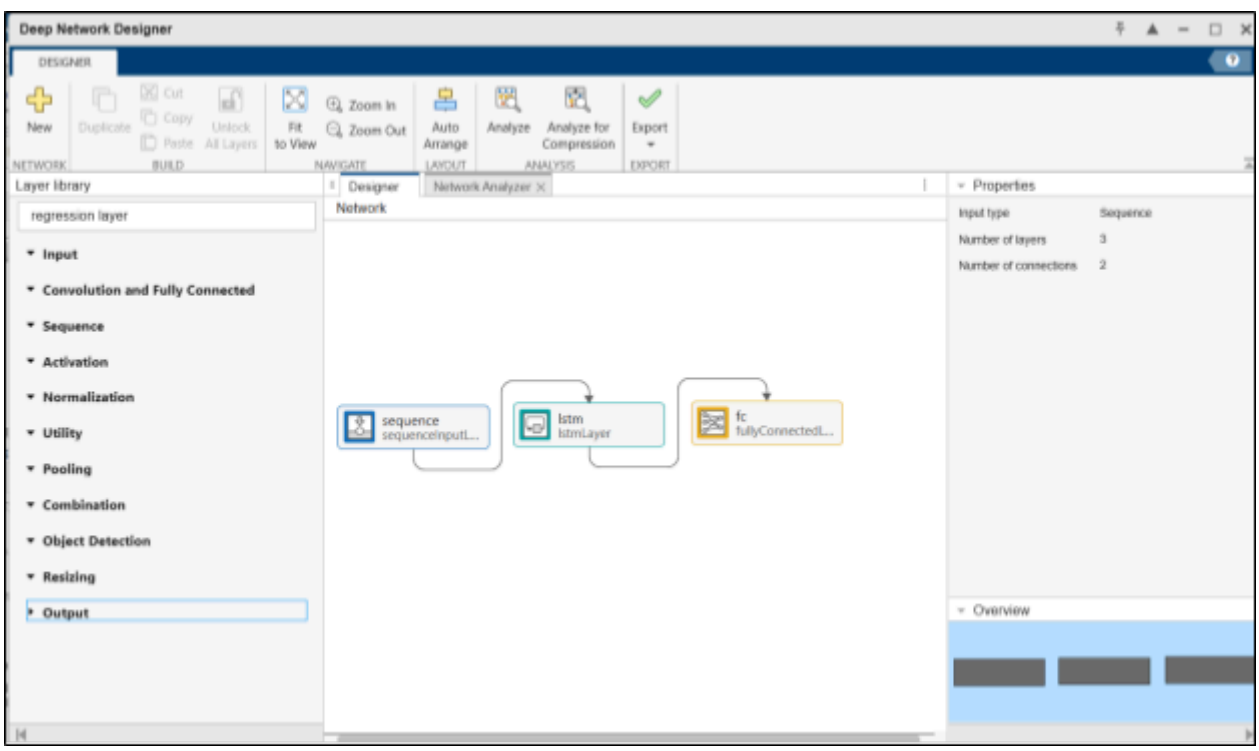
4. Convert data into sequence format (cell arrays)

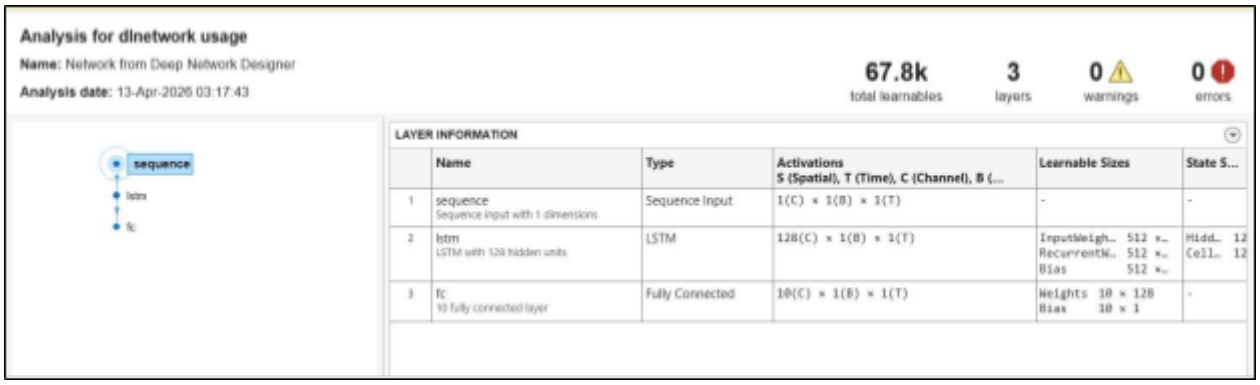
```
%% Step 4: Prepare Sequence Data
XTrain = num2cell(sunspots(1:end-1)');
YTrain = num2cell(sunspots(2:end)');
```

5. Open Deep Network Designer and create LSTM architecture

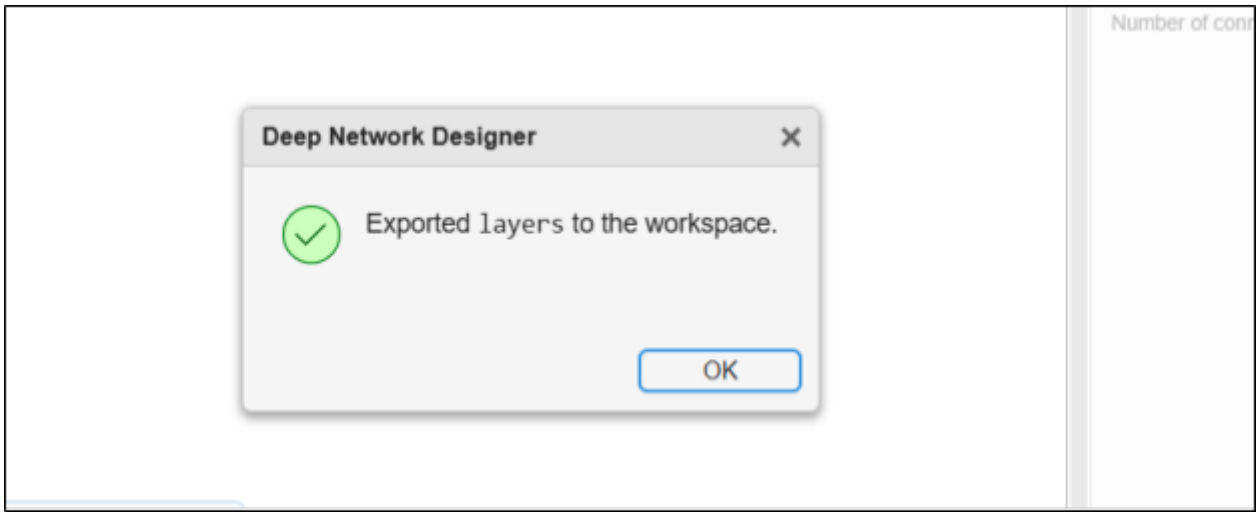



6. Add layers: Sequence Input → LSTM → Fully Connected






7. Export network to workspace




8. Add regression layer manually in script

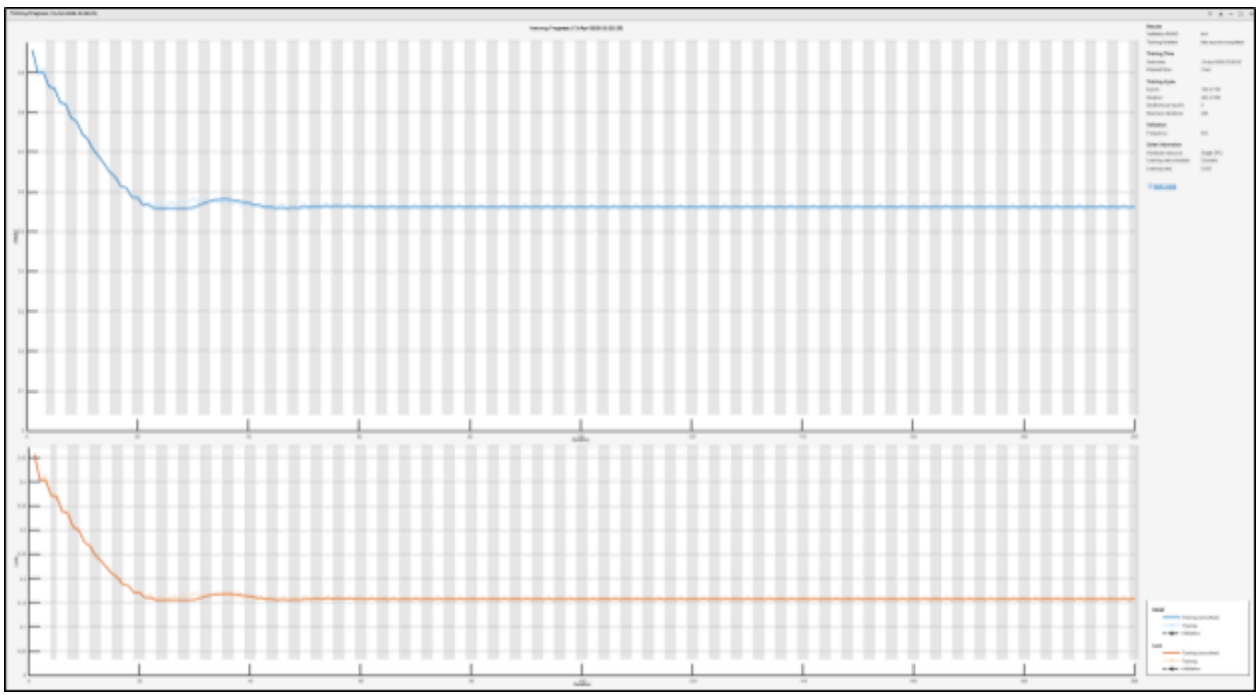
```
%% Step 5: Define LSTM Network
layers = [
    sequenceInputLayer(1)
    lstmLayer(100)
    fullyConnectedLayer(1)
    regressionLayer
];
```

9. Define training options

```
%% Step 5: Define LSTM Network
layers = [
    sequenceInputLayer(1)
    lstmLayer(100)
    fullyConnectedLayer(1)
    regressionLayer
];

```

10. Train the LSTM network

```
%% Step 7: Train Network
net = trainNetwork(XTrain, YTrain, layers, options);

```



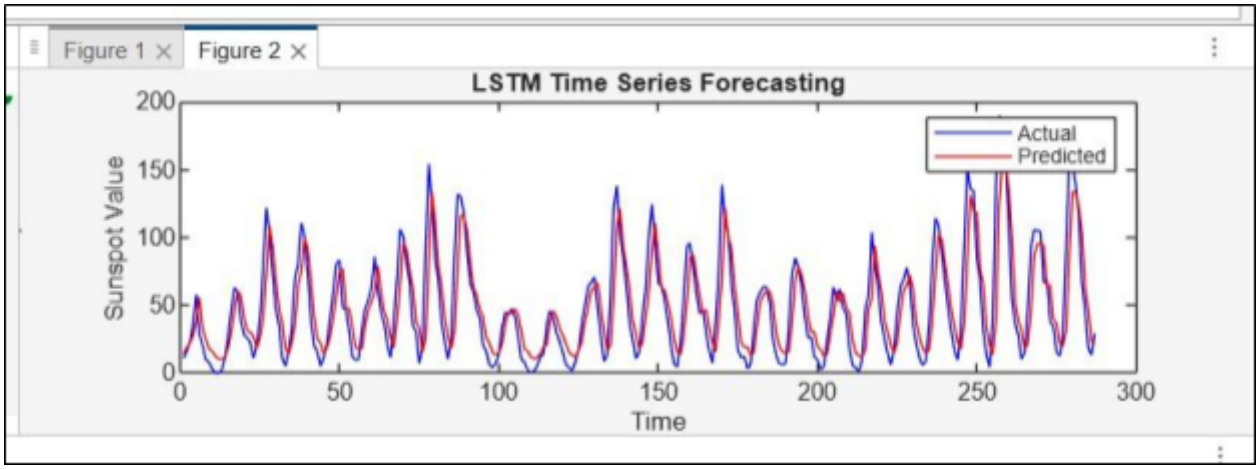
11. Predict future values

```
%% Step 8: Predict on Training Data
YPred = predict(net, XTrain);

```

```
%% Step 9: Convert Back to Original Scale
YPred = cell2mat(YPred) * sigma + mu;
YActual = cell2mat(YTrain) * sigma + mu;

```

12. Plot actual vs predicted values

```
%% Step 10: Plot Results
figure;
plot(YActual, 'b');
hold on;
plot(YPred, 'r');
legend('Actual','Predicted');
title('LSTM Time Series Forecasting');
xlabel('Time');
ylabel('Sunspot Value');
```




**Workflow Representation**

Dataset → Normalization → Sequence Conversion → LSTM Model → Training → Prediction → Evaluation

**5. Performance Analysis**

Evaluation Metrics

| Metric | Description |
|---|---|
| Loss | Training error |
| Visual Comparison | Actual vs predicted graph |

**Observed Results**

- Model successfully captures trend in time series
- Predicted values closely follow actual values

- Loss decreases during training

**Interpretation**

- LSTM effectively learns temporal dependencies
- Small deviations occur due to noise in data
- Model demonstrates good forecasting capabilit

## 6. Hyperparameter Tuning

**Parameters Tuned**

| Parameter | Values Tested |
|---|---|
| Hidden Units | 50, 100, 150 |
| Epochs | 50, 100 |
| Learning Rate | 0.005, 0.001 |
| Optimizer | Adam |

**Training Configuration**

```
options = trainingOptions('adam', ...
  'MaxEpochs',100, ...
  'GradientThreshold',1, ...
  'InitialLearnRate',0.005, ...
  'Verbose',0, ...
  'Plots','training-progress');
```

**Impact of Tuning**

- More hidden units improve learning capacity
- Higher epochs improve accuracy but risk overfitting
- Learning rate affects convergence speed
- Adam optimizer provides stable training

**Best Configuration**

- Hidden Units: 100
- Epochs: 100
- Learning Rate: 0.005
- Optimizer: Adam

## Conclusion

The LSTM network was successfully designed using the Deep Network Designer app and implemented in MATLAB for time series forecasting. The model effectively captured temporal dependencies in the dataset and produced accurate predictions. This experiment demonstrates the capability of LSTM networks in handling sequential data and forecasting future values.